Algoritmos Paralelos

Práctica 1: Instalación de OpenMP

Erick Joel Baltodano Cuevas

Fecha de entrega: 9/marzo/2026

1. Objetivos

- Instalar y configurar correctamente un entorno de desarrollo con soporte para OpenMP.
- Verificar la instalación mediante la compilación y ejecución de un programa básico en C que utilice directivas de paralelismo.
- Comprender el proceso de compilación con flags específicos para habilitar OpenMP.

2. Metodología

1. Se verificó la disponibilidad de un compilador con soporte OpenMP (gcc en Linux).
2. Se creó un programa simple en C que imprime un mensaje desde varias hebras paralelas.
3. Se compiló el programa usando el flag `-fopenmp` en gcc.
4. Se ejecutó el programa resultante y se observó la salida en consola.
5. Se tomó una captura de pantalla de la ejecución como evidencia.

3. Resultados

A continuación se muestra el código del programa utilizado para verificar la instalación:

Listing 1: hola_openmp.c

```c
#include <stdio.h>
#include <omp.h>

int main() {
    #pragma omp parallel
    {
        int id = omp_get_thread_num();
        printf("Hola desde la hebra %d\n", id);
    }
    return 0;
}
```

La compilación se realizó con el comando:

```
gcc -fopenmp hola_openmp.c -o hola_openmp
```

La ejecución produjo una salida similar a la siguiente (el orden de las hebras puede variar):

```
Hola desde la hebra 0
Hola desde la hebra 3
Hola desde la hebra 1
Hola desde la hebra 2
```

En las siguientes figuras se muestran ejecuciones con diferentes cantidades de hilos.

```
baltimus@Dev:~/semestre/paralelos/practicas/practica1$ export OMP_NUM_THREADS=4
./programa
bash: ./programa: No such file or directory
baltimus@Dev:~/semestre/paralelos/practicas/practica1$ ./a.out
Hola desde el hilo 2 de 4
Hola desde el hilo 1 de 4
Hola desde el hilo 3 de 4
Hola desde el hilo 0 de 4
baltimus@Dev:~/semestre/paralelos/practicas/practica1$
```

Figura 1: Ejecución del programa OpenMP con 4 hilos.

```
baltimus@Dev:~/semestre/paralelos/practicas/practica1$ export OMP_NUM_THREADS=19
baltimus@Dev:~/semestre/paralelos/practicas/practica1$ ./a.out
Hola desde el hilo 1 de 19
Hola desde el hilo 8 de 19
Hola desde el hilo 13 de 19
Hola desde el hilo 2 de 19
Hola desde el hilo 3 de 19
Hola desde el hilo 0 de 19
Hola desde el hilo 7 de 19
Hola desde el hilo 4 de 19
Hola desde el hilo 6 de 19
Hola desde el hilo 12 de 19
Hola desde el hilo 9 de 19
Hola desde el hilo 10 de 19
Hola desde el hilo 11 de 19
Hola desde el hilo 14 de 19
Hola desde el hilo 5 de 19
Hola desde el hilo 15 de 19
Hola desde el hilo 16 de 19
Hola desde el hilo 17 de 19
Hola desde el hilo 18 de 19
baltimus@Dev:~/semestre/paralelos/practicas/practica1$
```

Figura 2: Ejecución del programa OpenMP con 19 hilos.

```
baltimus@Dev:~/semestre/paralelos/practicas/practica1$ export OMP_NUM_THREADS=32
baltimus@Dev:~/semestre/paralelos/practicas/practica1$ ./a.out
Hola desde el hilo 2 de 32
Hola desde el hilo 31 de 32
Hola desde el hilo 30 de 32
Hola desde el hilo 29 de 32
Hola desde el hilo 25 de 32
Hola desde el hilo 24 de 32
Hola desde el hilo 28 de 32
Hola desde el hilo 26 de 32
Hola desde el hilo 23 de 32
Hola desde el hilo 22 de 32
Hola desde el hilo 21 de 32
Hola desde el hilo 20 de 32
Hola desde el hilo 19 de 32
Hola desde el hilo 18 de 32
Hola desde el hilo 17 de 32
Hola desde el hilo 16 de 32
Hola desde el hilo 15 de 32
Hola desde el hilo 14 de 32
Hola desde el hilo 13 de 32
Hola desde el hilo 12 de 32
Hola desde el hilo 11 de 32
Hola desde el hilo 10 de 32
Hola desde el hilo 9 de 32
Hola desde el hilo 8 de 32
Hola desde el hilo 7 de 32
Hola desde el hilo 6 de 32
Hola desde el hilo 4 de 32
Hola desde el hilo 1 de 32
Hola desde el hilo 5 de 32
Hola desde el hilo 3 de 32
Hola desde el hilo 0 de 32
Hola desde el hilo 27 de 32
baltimus@Dev:~/semestre/paralelos/practicas/practica1$
```

Figura 3: Ejecución del programa OpenMP con 32 hilos.

```
baltimus@Dev:~$ lscpu | grep "Core(s) per socket"
Core(s) per socket:                    2
baltimus@Dev:~$
```

Figura 4: Número de núcleos del sistema (verificado con el comando `lscpu`).

4. Conclusiones

- La instalación de OpenMP es sencilla cuando se dispone de un compilador como gcc, que ya incluye soporte nativo.
- El uso del flag `-fopenmp` es esencial para habilitar las directivas de paralelismo durante la compilación.
- La ejecución del programa de prueba confirmó que el entorno está correctamente configurado, ya que se imprimieron mensajes desde diferentes hebras.
- OpenMP permite paralelizar código de manera incremental con mínimos cambios en el código fuente, lo que facilita la transición de programas secuenciales a paralelos.
- Se verificó que el número de hilos puede configurarse mediante la variable de entorno `OMP_NUM_THREADS` o funciones como `omp_set_num_threads()`.
- El sistema utilizado cuenta con los núcleos mostrados en la figura, lo que permite ejecutar múltiples hilos de manera eficiente.